



GRAPHWORD

Arquitectura distribuida demostración AWS pruebas CI/CD y benchmark



7 DE OCTUBRE DE 2026

Grado en Ciencia e Ingeniería de Datos - Universidad de Las Palmas de Gran Canaria

Guillermo Cubas Granado

Contenido

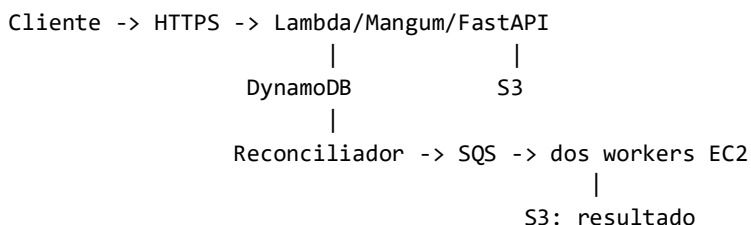
1	Objetivo y arquitectura implementada	2
2	Datos algoritmos y decisiones de diseño.....	3
	Diccionarios y construcción distribuida	3
	Consultas disponibles.....	3
	Robustez seguridad y operación	3
3	Demostración funcional de la API en AWS.....	4
	Acceso y ejemplo de consultas	4
	Construcción desde S3 y evidencia real	4
4	Informe de pruebas automáticas y CI/CD.....	5
	Integración continua	5
	Despliegue automatizado y alcance	5
5	Arquitectura empresarial: TOGAF y ArchiMate sin exagerar	6
	5.1 Qué se ha hecho	6
	5.2 Interesados	6
	5.3 Capa de aplicación	6
	5.4 Capa tecnológica.....	6
	5.5 Evolución según ADM adaptado.....	6
6	Benchmark de construcción del grafo	7
	Cómo hice la comparación en AWS	7
	Resultados observados	7

1 Objetivo y arquitectura implementada

GraphWord es un proyecto en Python que construye grafos de palabras y permite consultarlos mediante una API. Cada palabra es un nodo y se conecta con otra cuando tienen la misma longitud y difieren en una letra. El grafo es no dirigido y no ponderado.

He separado la aplicación en API, motor de grafos, workers, reconciliador y adaptadores de almacenamiento. El motor no depende de AWS, por lo que se puede probar localmente. La documentación distingue la vista de aplicación de la tecnológica, siguiendo una adaptación académica de TOGAF y conceptos de ArchiMate, sin afirmar conformidad formal.

Tecnología	Función y motivo de elección
FastAPI, Lambda y Mangum	API HTTPS con validación y Swagger. Mangum adapta FastAPI a Lambda sin rehacer las rutas
Dos EC2 t3.micro	Workers para repartir la construcción. Una aloja también el reconciliador y la API privada de respaldo
S3	Diccionarios, entradas, grafos parciales y finales compartidos entre procesos
DynamoDB	Registro duradero de trabajos, estados, reservas, intentos y dependencias
SQS y DLQ	Avisos a workers y mensajes con recepciones fallidas; separan petición y cálculo
CloudFormation y SSM	Recursos como código y administración de EC2 sin abrir SSH ni publicar el puerto 8000
GitHub Actions y runner EC2	Pruebas y despliegue automatizado. El runner es una tercera EC2, no un worker del grafo



2 Datos algoritmos y decisiones de diseño

Diccionarios y construcción distribuida

Los diccionarios combinan una fuente léxica Hunspell/SCOWL de LibreOffice y las pronunciaciones de CMUdict. Se conserva su intersección, filtrando letras ASCII minúsculas y longitudes de tres a ocho. Se eliminan duplicados y se registran fuentes, versiones, huellas y licencias. Es un filtro reproducible, no una garantía lingüística perfecta.

El catálogo y los archivos se publican en S3. `words3` contiene 628 palabras y produce 3815 aristas. La API permite seleccionar el diccionario por ID y verificar su integridad, o recibir una lista del usuario con normalización sintáctica.

Para encontrar conexiones genero patrones: `cat` y `bat` comparten `*at`. Reparto patrones mediante SHA256 y el número de particiones. No divido las palabras en bloques independientes, porque perdería conexiones entre bloques.

Con ocho particiones se crean ocho tareas y una reducción. Cada tarea lee las palabras y procesa solo sus patrones. Los dos workers recogen tareas disponibles sin un reparto fijo de cuatro para cada uno. Al terminar, un worker une los parciales guardados en S3.

Consultas disponibles

Operación	Implementación y decisión
Camino mínimo	BFS, porque todas las aristas cuestan uno
Todos los caminos	Exploración de caminos simples con límites de tiempo, profundidad, estados y resultados
Camino más largo	Mejor camino simple entre dos nodos; máximo demostrado solo si <code>complete=true</code>
Grados y aislados	Contar vecinos y seleccionar por grado, máximo grado o grado cero
Subgrafos densos	Regiones k-core con un grado interno mínimo; no se presenta como Louvain

Robustez seguridad y operación

DynamoDB controla la reserva de cada trabajo; SQS solo avisa. El heartbeat renueva la reserva mientras el worker calcula y el reconciliador recupera tareas y habilita la reducción. Una clave de idempotencia evita duplicar peticiones. Se admiten mensajes repetidos, pero no se promete ejecución exactamente una vez.

La entrada HTTPS exige contraseña y sesión firmada. Lambda guarda sal y hash, no la contraseña en texto. La Function URL admite invocación pública; la protección está en la aplicación, por lo que los rechazos también pueden consumir recursos. S3 es privado y los procesos usan los roles existentes del laboratorio, sin ampliar permisos.

Al terminar se pausa Lambda y se detienen por separado los workers y el runner. Los datos se conservan. No hay autoscaling ni garantía de alta disponibilidad: la reducción en memoria y el reconciliador único limitan la escala.

3 Demostración funcional de la API en AWS

Acceso y ejemplo de consultas

Abrir [Swagger HTTPS de GraphWord](#). Usuario: graphword. La contraseña se facilita por separado. Lambda debe estar reanudada; para trabajos distribuidos deben estar activos los dos workers y el reconciliador.

En Swagger se utiliza Try it out, Execute y la respuesta real del servidor. Primero se envía POST /v1/graphs:

```
{
  "words": ["cat", "bat", "bad", "dad", "mat", "rat", "zzz"],
  "partitions": 2
}
```

El resultado esperado es HTTP 201, siete nodos, ocho aristas y dos componentes. Se guarda el graph_id y se utiliza en las siguientes consultas:

Prueba	Entrada y resultado esperado
POST .../queries/shortest-path	start=cat, end=dad: cat, bat, bad, dad
POST .../queries/all-simple-paths	Mismos extremos: cinco caminos y complete=true en el ejemplo pequeño
POST .../queries/longest-simple-path	cat, mat, rat, bat, bad, dad; complete=true
GET .../nodes	degree=0 devuelve zzz; degree=4 devuelve bat
GET .../dense-subgraphs	minimum_degree=3 devuelve bat, cat, mat, rat

... representa /v1/graphs/{graph_id}. Para las búsquedas completas del ejemplo se usan 10000 estados, profundidad 10 y timeout de cinco segundos; all-simple-paths admite hasta 100 resultados.

Construcción desde S3 y evidencia real

Se consulta GET /v1/dictionaries y se envía POST /v1/jobs/dictionary-builds con {"dictionary_id":"words3","partitions":8}. HTTP 202 significa aceptado, no terminado. Se consulta GET /v1/jobs/{job_id} hasta SUCCEEDED y se obtiene result.graph_id. El resumen debe indicar 628 nodos, 3815 aristas y 12 componentes; el camino cat-dad es cat, cad, dad.

4 Informe de pruebas automáticas y CI/CD

Integración continua

GitHub Actions instala el proyecto y ejecuta unittest en Python 3.11 y 3.12. Los 122 casos cubren motor, API, persistencia, coordinación, diccionarios, seguridad y despliegue. Los tests del motor incluyen comparación con un constructor independiente de parejas de palabras.

Moto simula S3, DynamoDB y SQS para pruebas repetibles sin desplegar recursos. No es AWS real ni LocalStack. Los smoke de despliegue verifican AWS por separado. La aprobación de todos los casos no equivale a cobertura completa de líneas o ramas.

Entorno	Casos	Resultado	Tiempo unittest
Ubuntu 22.04 / Python 3.11	122	OK	11,518 s
Ubuntu 22.04 / Python 3.12	122	OK	12,843 s

Son las mismas 122 pruebas en dos intérpretes, no 244 casos distintos. Los artefactos `unittest-3.11` y `unittest-3.12` conservan los registros. El comando reproducible es:

```
python -m unittest discover -s tests -v
```

Despliegue automatizado y alcance

Tras aprobar CI, el CD autorizado utiliza un runner EC2 con rol temporal, descarga el commit probado, publica el release y aplica CloudFormation. Comprueba API y workers mediante SSM y finalmente solicita y verifica la parada de las EC2 de aplicación. Requiere repositorio privado, rama autorizada y `ENABLE_LAB_CD=true`; no despliega pull requests ni forks.

5 Arquitectura empresarial: TOGAF y ArchiMate sin exagerar

5.1 Qué se ha hecho

La documentación adapta el método ADM de TOGAF al tamaño del proyecto y utiliza conceptos de capas y relaciones de ArchiMate para ordenar las vistas. Se declara expresamente que no es una certificación formal ni un modelo ArchiMate validado por una herramienta.

5.2 Interesados

Interesado	Necesidad principal
Estudiante desarrollador/operador	Implementar, demostrar, repetir y controlar coste
Profesor evaluador	Ver corrección, diseño, trazabilidad y evidencia
Usuario analista	Construir y consultar grafos mediante la API

5.3 Capa de aplicación

Elemento	Responsabilidad
A1 API	Recibir construcciones y consultas
A2 Motor	Algoritmos de grafo
A3 Reconciliador	Recuperar leases, publicar pendientes, habilitar reducción
A4 Worker	Reclamar, renovar, calcular, guardar y confirmar
A5 Adaptadores	Persistencia/concurrencia en memoria, SQLite o AWS
A6 Preparación de corpus	Filtrado, intersección, manifiesto y licencias

5.4 Capa tecnológica

Lambda publica la API HTTPS; EC2 ejecuta componentes; S3 conserva datos; DynamoDB coordina estados; SQS avisa; SSM permite administración; CloudFormation define infraestructura; CloudWatch conserva registros de ejecución; GitHub Actions verifica el código.

5.5 Evolución según ADM adaptado

La historia técnica se presenta como transición progresiva: motor comprobable → API → persistencia local → procesos separados → particiones → servicios AWS compartidos → hardening → medición. Cada etapa elimina una limitación concreta sin fingir que la siguiente ya existía antes.

6 Benchmark de construcción del grafo

Cómo hice la comparación en AWS

Comparé uno y dos workers EC2 t3.micro en AWS. Usé diccionarios de tres a seis letras, ocho particiones y tres repeticiones por configuración: 4 diccionarios x 2 configuraciones x 3 repeticiones = 24 construcciones. Alterné el orden de uno y dos workers para reducir el efecto de las cachés.

Medí desde el envío HTTP a la API privada EC2 hasta observar el trabajo en SUCCEEDED. El tiempo incluye S3, DynamoDB, SQS, workers, reducción y consultas de estado.

Resultados observados

Los tiempos son medianas de tres repeticiones, en segundos. La mejora se calcula como tiempo con un worker dividido entre tiempo con dos; un valor mayor que uno indica una construcción más rápida.

Diccionario	Nodos	Aristas	1 worker (s)	2 workers (s)	Mejora
words3	628	3815	1,966	1,417	1,39 veces
words4	1889	7153	1,987	1,424	1,40 veces
words5	2677	3171	2,158	1,481	1,46 veces
words6	3676	2332	2,084	1,498	1,39 veces

Con dos workers, la mediana del tiempo se redujo entre un 28 % y un 31 %. Los 24 grafos coincidieron con el constructor de referencia. Aumentar la longitud de las palabras no implica aumentar las aristas: en este corpus, words4 tiene más conexiones que words6.